

Predator

A Trainable Automated Theorem Prover

Measuring a Learned Policy Against Brute Force

Without Rewarding It for Giving Up

Version 1.1

Brian Tenneson
M.S., Applied Data Science
Currently Unaffiliated
email: btenneson2301@baypath.edu

July 27, 2026

Predator-ATP-Brian_Tenneson_1.1 | Version 1.1 | July 27, 2026

Companion program: `predator.py`

Copyright © 2026 Brian Tenneson. All Rights Reserved.

<https://btenneson2301.substack.com>

Abstract

Predator is a tagged automated theorem prover that learns its search policy from proofs. It is a semi-ideal computer over a formal system $F = (x, y, z)$, carrying a designated formal name $\langle \text{Predator_1} \rangle$, and it differs from the canonical rule-enumeration machine in exactly one respect: where brute force adjoins every direct consequence at each stage, Predator scores the available consequences with a fitted model and adjoins only the best few.

The evaluation is the substance of this paper. A prover is trained on the first p fraction of a corpus and tested on the remaining $1 - p$, split by corpus order so that training never sees a theorem postdating a test target. The natural way to score it — how far its search effort falls below brute force — is unusable on its own, because a prover that abandons a target expands almost nothing and so scores perfectly. Predator is therefore scored on two numbers that cannot be optimized separately: the *solve rate* over all held-out targets, and the *effort ratio* over targets that brute force and Predator both solved.

The distinction is not hypothetical. At $p = 0.3$ Predator records an effort ratio of 0.172, an apparent speedup of $11.6\times$, while solving only 70% of held-out targets: a single-number score would rate it a success. At $p = 0.9$ it solves every target at a ratio of 0.085, a genuine $14.7\times$. Only the paired reporting distinguishes these.

A third quantity is checked rather than optimized. By the depth-floor theorem no policy can reach a target at a stage below its closure depth, so the harness records any breach as an implementation fault. None occurred in any run reported here.

This version separates two things the first conflated. *Expansions* — formulas adjoined before the target appears — are hardware independent, and on them Predator beats brute force by

roughly $17\times$. *Wall clock* is what a user waits, and on it Predator is currently about twice as *slow*, because it scores every candidate it then declines to keep and so pays about $33\times$ more per expansion. Reporting the first without the second overstates the prover. The gap does narrow with difficulty — brute-force time grows about $3.9\times$ per unit of target depth against Predator’s $2.4\times$ — which locates the regime where training amortizes and is the subject of [Section 5](#).

Keywords: automated theorem proving, learned search policy, beam search, brute force baseline, solve rate, effort ratio, temporal split, tagged ATP, closure depth, semi-ideal computer.

Contents

1 What Predator is	4
2 Training	4
3 Scoring a prover	5
4 Results	5
5 Expansions, seconds, and when training pays	6
5.1 Two different quantities	6
5.2 Break-even	6
5.3 What changes with difficulty	7
5.4 What Predator learns, in pictures	7
6 Scope	7
7 Using it	10
Acknowledgements	10

1 What Predator is

Definition 1.1 (Predator as a tagged ATP). Fix a formal system $F = (x, y, z)$ and hypothesis set Γ . *Predator* is the pair (C, H) over Y given by

$$C(\Gamma, 1) = \Gamma, \quad C(\Gamma, m + 1) = C(\Gamma, m) \cup \text{top}_\beta(\sigma_\varphi, D_F(C(\Gamma, m)) \setminus C(\Gamma, m)),$$

$$H(\Gamma, m) = 1 \iff \varphi \in C(\Gamma, m),$$

where φ is the target, σ_φ is a learned score on candidate formulas, β is the beam width, and top_β selects the β highest-scoring candidates. Predator carries a designated formal name $\langle \text{Predator_1} \rangle$ in the sense of the tagging framework.

Proposition 1.2. *Predator is a semi-ideal computer, and it is a rule-driven target-search ATP for φ whenever $\beta \geq 1$.*

Proof. Stage one is the input by construction. The stage map is nondecreasing because each stage adjoins a set to the previous state. Halting is persistent, since $\varphi \in C(\Gamma, m)$ and monotonicity give $\varphi \in C(\Gamma, m + 1)$; and the state freezes on halting if the search stops on success. Rule-drivenness holds because every formula adjoined lies in $D_F(C(\Gamma, m))$, hence is a direct consequence of the current state. Clause (1) of the target-search definition is immediate from the form of H . $\square$

Remark 1.3 (Where the learning lives, and what it cannot touch). Predator is a nondeterministic proof-search policy whose choice function was fitted to data rather than fixed. Its only departure from the canonical machine is the top_β operator: brute force takes all of $D_F(C(\Gamma, m))$, Predator takes β of it. Everything provable about policies therefore applies, in particular the depth floor $p_m \subseteq D_F^{m-1}(\Gamma)$: Predator can reduce how much of each closure round it materializes, and cannot reduce how many rounds are required. Discarding candidates is also why Predator can fail where brute force succeeds, which is the whole reason solve rate must be reported.

2 Training

Definition 2.1 (Split). Order the corpus by the order in which statements were derived. For $p \in (0, 1)$, the first $\lfloor pN \rfloor$ theorems are training data and the rest are held out. The split is by corpus order, not at random: a random split would let the model train on theorems postdating its test targets, which is the usual way such numbers are inflated.

Definition 2.2 (Supervision). For a training target τ , the positive examples are the statements in $\text{anc}(\tau)$, those actually used in its proof; the negatives are statements available before τ that its proof did not use. The model therefore learns to recognize *this is on the way to that*, which is the judgement a search policy has to make at every step.

The score is a logistic model on eight cheap syntactic features of a pair (goal, candidate): token overlap with the goal, the share of the candidate the goal covers, whether the candidate occurs literally in the goal, size mismatch, candidate size, the candidate’s closure depth, and whether the candidate is no larger than the goal. Features are standardized before fitting and the training transform is reapplied at scoring, never refitted on the test set.

3 Scoring a prover

Remark 3.1 (Why one number will not do). The intuitive score is the distance of Predator’s search effort below brute force. As a sole criterion it is unusable, because it is maximized by failure: a prover that expands three nodes and abandons the target has an effort ratio near zero. Any single-number version of “further from brute force is better” ranks a prover that gives up above one that works.

Definition 3.2 (The two numbers). Let T be the held-out targets, and for each $t \in T$ let $e_{\text{bf}}(t)$ and $e_{\text{pr}}(t)$ be the number of formulas each prover adjoins before reaching t , within a fixed node budget. Write S_{bf} and S_{pr} for the targets each solves. Then

$$\text{solve rate} = \frac{|S_{\text{pr}}|}{|T|}, \quad \text{effort ratio} = \text{mean}_{t \in S_{\text{bf}} \cap S_{\text{pr}}} \frac{e_{\text{pr}}(t)}{e_{\text{bf}}(t)}.$$

The ratio is taken over *jointly solved* targets only. Restricting it there is what stops abandonment from counting as efficiency, and reporting it without the solve rate beside it restores the defect.

Definition 3.3 (The floor check). For each jointly solved target the harness compares the stage at which each prover halts. By the depth floor a policy cannot halt earlier than $\delta_F(\Gamma, \varphi) + 1$, so a run in which Predator halts at a strictly earlier stage than brute force indicates a fault — most often a target reachable by a rule the baseline does not implement. This is checked, not optimized.

4 Results

The corpus is generated by running the canonical machine on a propositional Hilbert system with modus ponens and two axiom schemes read as generating rules: 789 statements over 6 stages, with stage sizes 3, 39, 111, 381, 579, 789. Beam width 8, node budget 60,000, 60 held-out targets of closure depth at least 2.

p	train	held out	brute force solve rate	Predator solve rate	jointly solved	effort ratio mean / median	speedup
0.3	235	551	100%	70.0%	42	0.172 / 0.115	11.6×
0.5	393	393	100%	76.7%	46	0.132 / 0.101	11.4×
0.7	550	236	100%	100%	60	0.133 / 0.095	10.7×
0.8	628	158	100%	100%	60	0.115 / 0.087	12.5×
0.9	707	79	100%	100%	60	0.085 / 0.067	14.7×

Depth-floor breaches: 0 in every row.

Remark 4.1 (The metric earning its keep). The first two rows are the argument for Definition 3.2. At $p = 0.3$ Predator’s effort ratio is 0.172 and its apparent speedup 11.6×

 — numbers that, alone, describe a working prover. Its solve rate is 70%. The ratio is respectable precisely because it is computed over the 42 targets Predator did not abandon; the 18 it walked away from contribute nothing to it. A single-number score would have reported the $p = 0.3$ model as roughly as good as the $p = 0.9$ model, when in fact one solves every target and the other misses three in ten.

Remark 4.2 (More training data helps twice). Solve rate rises from 70% to 100% between $p = 0.3$ and $p = 0.7$ and stays there; the effort ratio then continues to improve, from 0.133 at $p = 0.7$ to 0.085 at $p = 0.9$. The two gains are distinct: the first is the model learning to keep the right candidates at all, the second is its learning to rank them higher once kept. Note that held-out set size shrinks as p grows, so the $p = 0.9$ row rests on 79 available targets; the improvement is real but is measured on less.

5 Expansions, seconds, and when training pays

5.1 Two different quantities

Definition 5.1 (Expansion). An *expansion* is one formula adjoined to the prover’s state. Brute force expands every member of $D_F(C(\Gamma, m)) \setminus C(\Gamma, m)$ at each stage; Predator expands the β it scores highest. The expansion count to a target is the number of formulas adjoined before it appears.

Expansions are the right unit for comparing *policies*: they do not depend on the machine, the language, or how carefully anything was implemented. They are what the theory bounds, since the depth floor is a statement about which sets p_m can be, not about how long computing them takes.

Definition 5.2 (Wall clock). The *wall clock* of a proof is elapsed real time. For N proofs the total cost of a learned prover is

$$T_{\text{pr}}(N) = t_{\text{train}} + N t_{\text{pr}}, \quad T_{\text{bf}}(N) = N t_{\text{bf}},$$

where t_{train} is paid once.

Remark 5.3 (Why the two disagree). Predator scores *every* candidate at each stage and keeps β of them. The candidates it discards were still generated and still featurised, so the saving is not in the current stage but in the next one: a smaller state produces fewer combinations. Against that, each expansion costs far more. Measured over 20 held-out targets:

$$\text{expansions } 17.6 \times \text{ fewer,} \quad \text{cost per expansion } 33 \times \text{ higher,} \quad \text{net } 0.48 \times \text{ (slower).}$$

A prover can therefore be much better on the quantity the theory bounds and worse on the quantity a user experiences. Both must be reported.

5.2 Break-even

Proposition 5.4 (When training amortizes). *Training pays for itself after*

$$N^* = \frac{t_{\text{train}}}{t_{\text{bf}} - t_{\text{pr}}}$$

proofs, and only if $t_{\text{pr}} < t_{\text{bf}}$. If the learned prover is slower per proof, no volume of work recovers the training cost: the two cost lines diverge rather than crossing.

Proof. $T_{\text{pr}}(N) < T_{\text{bf}}(N)$ iff $t_{\text{train}} < N(t_{\text{bf}} - t_{\text{pr}})$. If the bracket is positive, divide; if it is non-positive the inequality fails for every $N \geq 0$. $\square$

Remark 5.5 (The present position, stated plainly). At the scale measured here $t_{\text{pr}} \approx 57$ ms against $t_{\text{bf}} \approx 29$ ms, so the bracket is negative and N^* does not exist. Training costs about 0.4s and never amortizes, because there is no per-proof saving to amortize against. Figure 4 shows the two lines diverging.

5.3 What changes with difficulty

The interesting question is not whether Predator wins now but whether the bracket in Proposition 5.4 turns positive as targets get harder. Brute force must materialize an entire closure round; Predator scores a round and keeps a fixed number. As depth grows the first is combinatorial and the second is linear in the candidates offered.

target depth	brute expansions	Predator expansions	ratio	brute ms	Predator ms
5	741	45	0.061	11.9	33.8
6	890	57	0.064	46.7	81.1
growth per level	1.20×	1.27×	—	3.92×	2.40×

Remark 5.6 (An extrapolation, offered as a prediction). Wall-clock time grows by about 3.92× per unit of depth for brute force and 2.40× for Predator. Continuing both geometrically, they meet where $11.9 \cdot 3.92^k = 33.8 \cdot 2.40^k$, that is at $k \approx 2.1$, so at target depth near 7. Predator would then be faster in seconds as well as in expansions, the bracket would turn positive, and N^* would become finite.

This rests on two points and a geometric assumption, which is thin. It is recorded as a falsifiable prediction rather than a result: run the harness at depths 7 and 8 and the crossover either appears near there or it does not. The honest summary of the present position is that Predator wins on the quantity the theory bounds, loses on the clock, and that the loss is shrinking with difficulty at a rate that would eliminate it in the regime where proof search is actually hard.

Remark 5.7 (What “ideal” means here). Given a fixed corpus and a workload of N theorems, the configuration to prefer maximises expansions saved per unit of scoring cost. Three levers exist. The beam width β trades solve rate against effort and should be set at the smallest value that keeps the solve rate at ceiling. The training fraction p raises both solve rate and effort ratio up to about $p = 0.7$, beyond which only the ratio improves. And the scoring cost — currently 668 μ s per expansion of unvectorised Python feature construction — is implementation, not method: batching the featurisation across a stage would cut it substantially and move the crossover of Remark 5.6 to shallower depths. Of the three, the last is where the leverage is.

5.4 What Predator learns, in pictures

6 Scope

The corpus is a propositional system of 789 statements generated from three atoms under three rules, and its regularity far exceeds real mathematics. The features are eight hand-built syntactic quantities under logistic regression. The baseline is unguided closure, which is the correct benchmark for this comparison but is a low bar in absolute terms. Nothing here is a result about theorem proving; it is evidence that the prover, the split and the scoring behave as designed, on data where ground truth is exactly known.

The next step is the same harness against a Metamath corpus, where premises are real, proofs are human-written, and lemma reuse means proof lengths are measured in a compiled system rather than the base one.

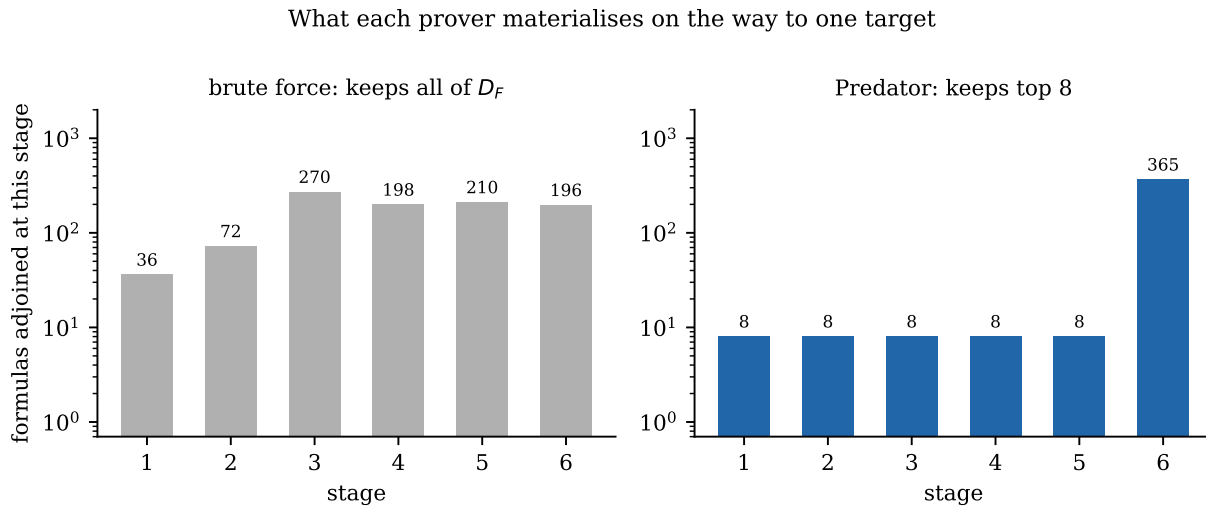


Figure 1: The two search frontiers on one target, by stage, on a logarithmic axis. Brute force adjoins all of D_F at each round and its state grows into the hundreds; Predator adjoins at most $\beta = 8$. The difference in area under these bars is the effort ratio.

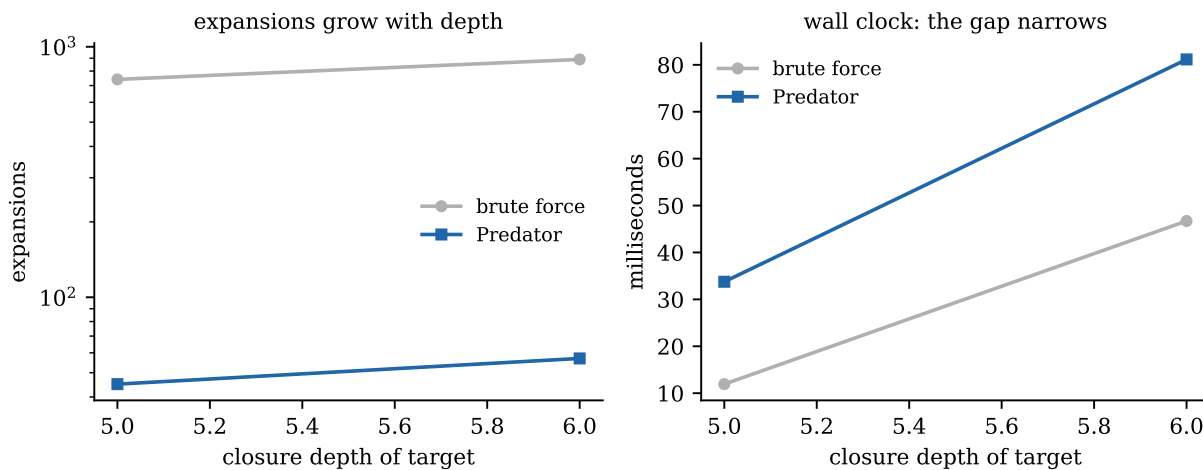


Figure 2: Left: expansions against target depth, logarithmic. The gap is roughly constant in ratio. Right: the same runs in milliseconds. Brute force starts far cheaper per proof and climbs faster, which is the trend Remark 5.6 extrapolates.

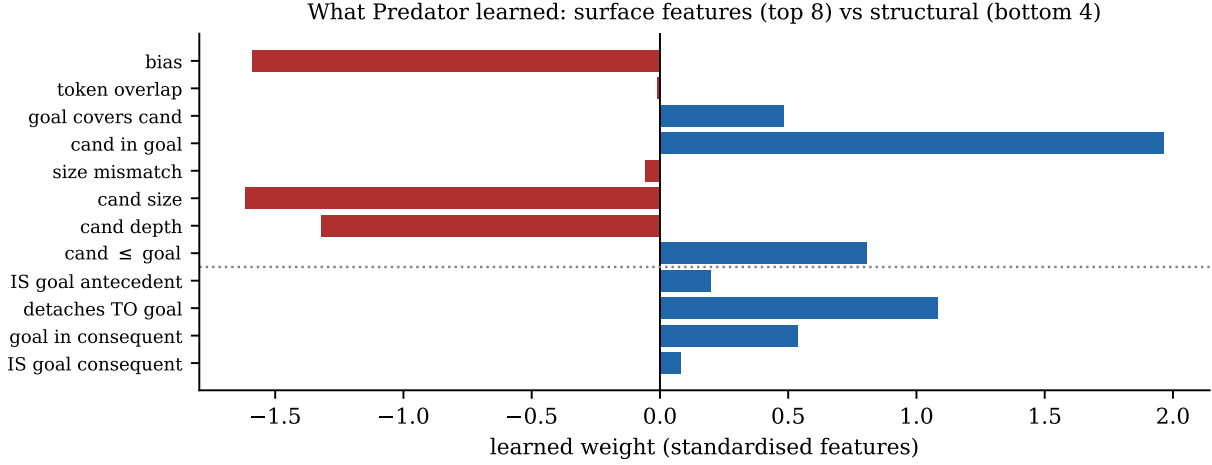


Figure 3: Learned weights on standardised features. The heaviest single weight is still “candidate occurs literally in the goal”, which on this corpus largely identifies instances of the weakening scheme $A \rightarrow (B \rightarrow A)$ — Predator has learned one axiom’s silhouette. The structural feature “detaches to the goal” carries the second-largest positive weight, so the modus ponens relation is being used as well; it was added after the surface-only model was inspected.

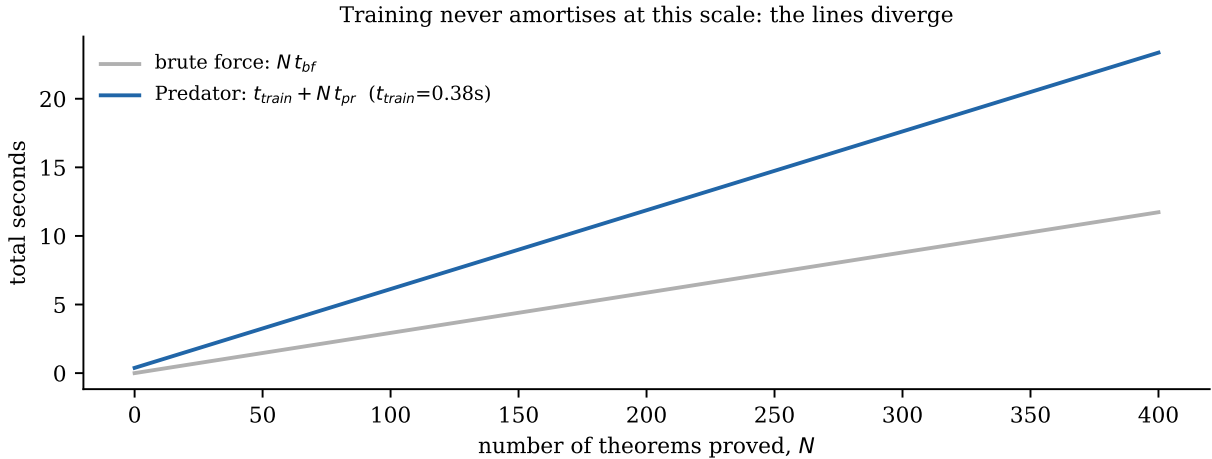


Figure 4: Total cost against number of theorems proved. The learned prover starts behind by t_{train} and, at this scale, never catches up: its slope is the larger. A finite break-even point exists only when the Predator line is the shallower one.

7 Using it

Windows PowerShell.

```
# train on 90% of the corpus, evaluate on the rest
python predator.py -p 0.9 --beam 8 --n-test 60 --budget 60000

# send every run to a chosen folder; a subfolder stamped to the minute
# is created inside it, so nothing is overwritten
python predator.py -p 0.9 --outdir "C:\google drive\Automated Theorem Proving"

# set it once for the session instead of repeating the flag
$env:PREDATOR_OUT = "C:\google drive\Automated Theorem Proving"

# sweep the training fraction -- PowerShell, not bash
foreach ($p in 0.3,0.5,0.7,0.9) { python predator.py -p $p --no-artifacts }
```

macOS and Linux.

```
python3 predator.py -p 0.9 --beam 8 --n-test 60 --budget 60000
export PREDATOR_OUT="$HOME/Automated Theorem Proving"
for p in 0.3 0.5 0.7 0.9; do python3 predator.py -p $p --no-artifacts; done
```

Flags: `-p` training fraction; `-beam` how many consequences to keep per stage; `-n-test` held-out targets to attempt; `-budget` node cap per proof; `-n-neg` negatives per training target; `-outdir` where run folders go; `-no-artifacts` to print without writing. Paths containing spaces must be quoted, and work.

Each run writes a directory stamped to the minute containing `manifest.json`, `run.log`, `results.json`, `nodes.csv`, `edges.csv` and `predator_1.json` — the last being the fitted prover itself: tag, beam width, weights and the standardization it was trained under. That file is the tagged copy: a serialized name for the machine, reloadable with `Predator.from_dict`, and small enough to print.

Remark 7.1 (A note on the tagged copy). Storing the prover as `<Predator_1>` together with its weights makes the self-reference machinery available: a copy is a syntactic object denoting the machine, transportable and comparable, and two Predators trained on different corpora can be compared as objects rather than only by their scores. Nothing in this paper needs that, but it is why the serialization records the tag rather than only the parameters.

Acknowledgements

The author thanks AI assistants used as engineering and verification tools during preparation, in particular for identifying that a single-number distance-from-brute-force criterion is maximized by abandoning targets, and for locating a generation-order mismatch that had made better than half the corpus appear underivable. All claims, interpretations and final decisions remain the responsibility of the author.

References

- [1] Tenneson, B. (2026). *ML-SIC-ATP Theory: Learned Search Policies for Proof Search*, Version 1.1.
- [2] Tenneson, B. (2026). *Depths of a Simulation and Formalized Self-Awareness*, Merged Edition.

- [3] Megill, N. D., & Wheeler, D. A. (2019). *Metamath: A Computer Language for Mathematical Proofs* (2nd ed.). Lulu Press.
- [4] Pearl, J. (1984). *Heuristics: Intelligent Search Strategies for Computer Problem Solving*. Addison-Wesley.